Rowan University: Team Ducks

Course Scheduler

Design Document

Prepared by: ***Steven Jiang***

Version: 1.1

Date: ***03/31/2020***

Sponsor: ***Dr. Vasil Hnatyshin***

Table of Contents

1. Introduction 3
    1.1. Purpose 3
    1.2. Scope 3
    1.3. System Environment 3
    1.4. User Interface Design 4
2. Architectural Design Views 5
    2.1. Logical View 5
    2.2. Development View 6
    2.3. Process View 7
    2.4. Physical View 8
3. Use Case Diagram 9
4. Analysis Class Diagrams 10
    4.1. General User 10
    4.2. System Creator 11
5. Statechart Diagram 12
6. Class Diagram 12
7. Activity Diagram 13
8. Sequence Diagrams 14
    8.1. Assign Course Section 13
    8.2. Assign Instructor 14
    8.3. Automatic Suggestion of Course 14
    8.4. Generate Report 15
    8.5. Update Database 15
9. Entity-Relationship Diagram 17
10. Revision History 18

1. Introduction

1.1. Purpose

This Design Document will provide comprehensive details about the system architecture and software design that is currently present in the ***Course Scheduler*** application. Narrative and graphical documentation will be used to illustrate such design through Unified Modeling Language (UML) diagrams.

1.2. Scope

The main goal of the ***Course Scheduler*** application is to allow individual users to efficiently schedule university courses for a specific department for a specific semester. We are taking the tedious and error-prone process of scheduling and creating an application that modifies that process to be simple and time-saving. The ***Course Scheduler*** application is being developed for the Computer Science department at Rowan University but may be extended for usage in other departments as well.

1.3. System Environment

1.3.1.

The required Operating System is Windows because this project is specifically being built to work with Microsoft Excel. We are not certain how other Operating Systems will behave in the current of development.

1.3.2.

Both the current MySQL and local server version is 5.7.24. It is strongly emphasized that, once the product is put into production, the database will be residing on our client-side server; this means, that it will be local and in the computer of our customer.

1.3.3.

The programming language of choice is C# because it provides many well-documented libraries that are extremely suited to automate Microsoft Excel.

1.4. User Interface Design

This section will provide a quick snapshot of how we anticipate our primary Excel interface to look like within the ***Course Scheduler*** application. Note that the interface is still being developed, so **Figure 1.1** below will change and adapt further on in our development cycle.

Robinson 323 (30 seats)			
	Monday	Instructor	Notes for User
8:00 AM	IOOP	Chien	1
9:30 AM		Chien	
11:00 AM		Myers	
12:30 PM		Provine	
		Tinkham	
2:00 PM			
3:30 PM			
5:00 PM			
6:30 PM			
8:00 PM			
9:30 PM			

Figure 1.1 - Users may choose to manually type course/instructor names or select them from a dropdown list. There is also a column for user notes such as noting the section number for the course.

2. Architectural Design Views

The following Architectural Design View Diagrams describe the ***Course Scheduler*** application from the viewpoint of different stakeholders, such as end-users, developers, system engineers, and project managers. When all stakeholders, of a project, clearly understand the system structure and model, concerns and issues will be solved which leads to a better product. These four view models illustrate the architecture of our system.

2.1. Logical View

The following Logical View shows key abstraction in the system as objects. **Figure 2.1** below shows those objects: Excel Interface, C# Console, Database. This diagram provides a basic understanding of the structure and organization of the system. It is a high-level introduction; specific details will be introduced later on in the document.

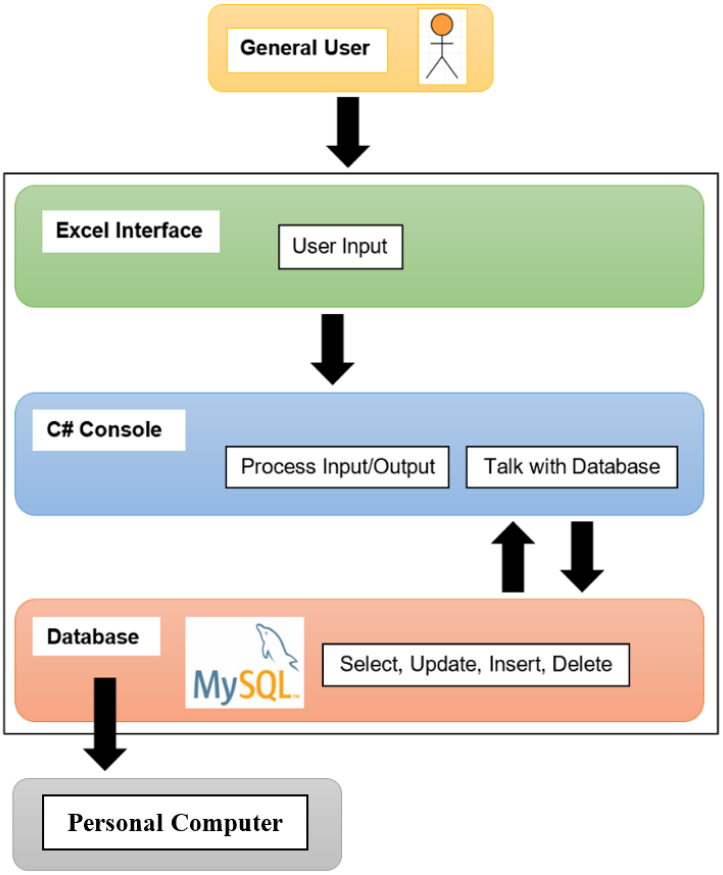

Figure 2.1

2.2. Development View

The following Development View shows how the software is decomposed into components for development. **Figure 2.2** below is a simple Component Diagram to show how the ***Course Scheduler*** was broken down into the Model-View-Controller (MVC). The MVC is a software design pattern that separates the program into organized components. The Mode contains application data; the View presents those data to the user; the Controller contains all the logic, where it manipulates the data from the Model and updates it to the View. The main interface for end-users will be the Excel spreadsheet which fits the View portion. User inputs and decisions will be taken in and processed in the Controller, C# classes. It facilitates the Model, which is the database, based on the user decision. The nature of the MVC components made dividing up jobs for the development team very suitable.

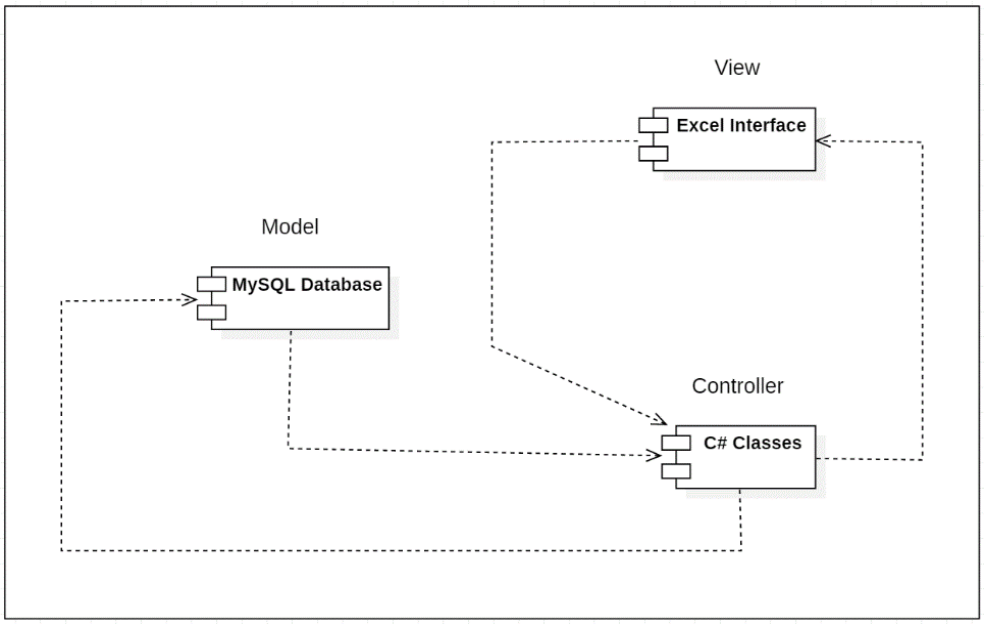

Figure 2.2

2.3. Process View

The following Process View shows the interactive processes of the system at runtime. **Figure 2.3** below is a Communication Diagram showing how objects interact in the ***Course Scheduler*** application. It shows the process flow when a General User begins to schedule courses, at a high-level in the Excel Sheets, all the way down to the low-level processes in the database.

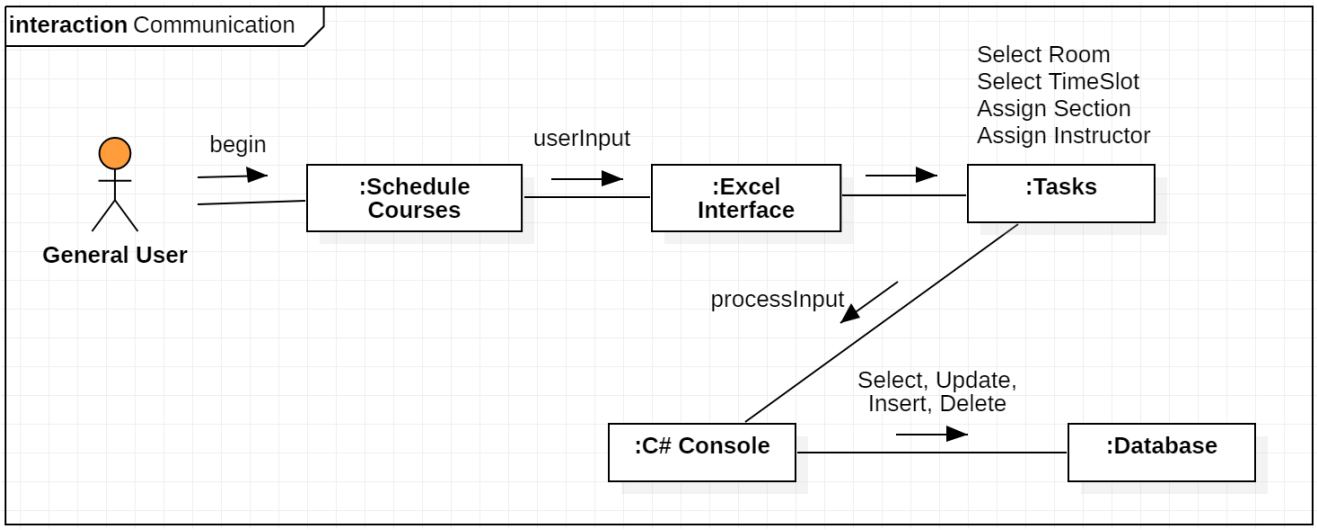

Figure 2.3

2.4. Physical View

The following Physical View shows the system from a system engineer's perspective. It shows system hardware and how software components are distributed across the hardware. **Figure 2.4** below is a Deployment Diagram which are great for visualizing the topology of the physical components of our ***Course Scheduler*** application. During the time that the project is in development for testing purposes, our database resides on Elvis, Rowan University's Sun Solaris based server that supports Unix. However, once the project is finished, per our sponsor's request, the database will then reside in a local server on their own personal computer. The main program, written in C#, will serve as the main bridge between the Excel interface and the database.

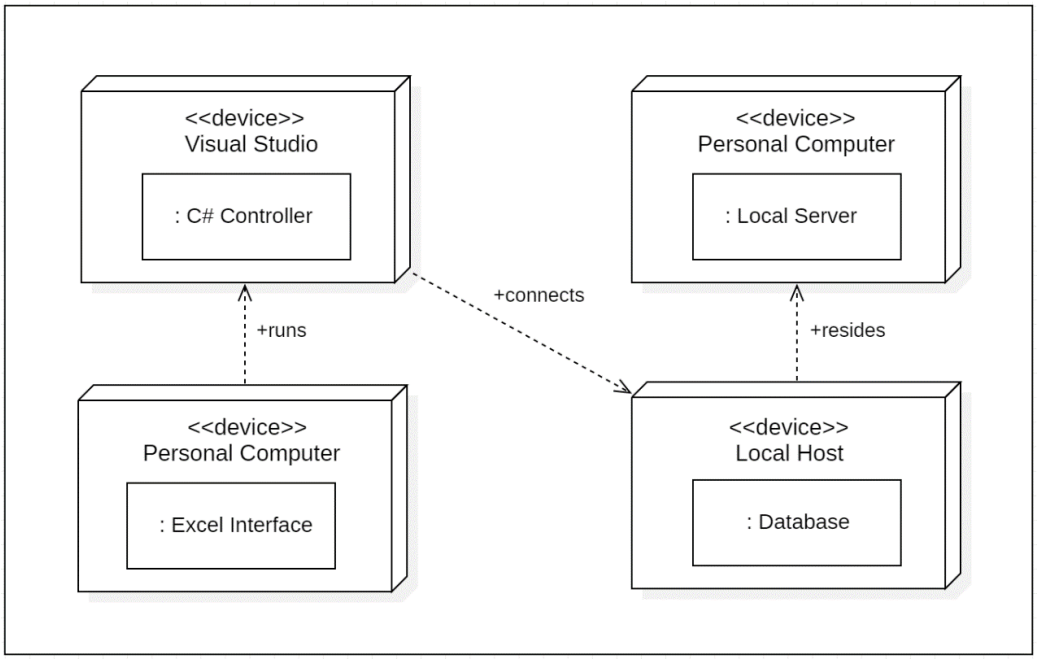

Figure 2.4

3. Use Case Diagram

Actor(s): General User, System Creator

The following Use Case Diagram shows all the actions that the Actor(s) above can perform while using the ***Course Scheduler*** application. **Figure 3.1** below helps visualize the different types of roles and show how these roles interact within our system. A General User can schedule courses; A System Creator can schedule courses as well, but they will be maintaining and updating the system mostly.

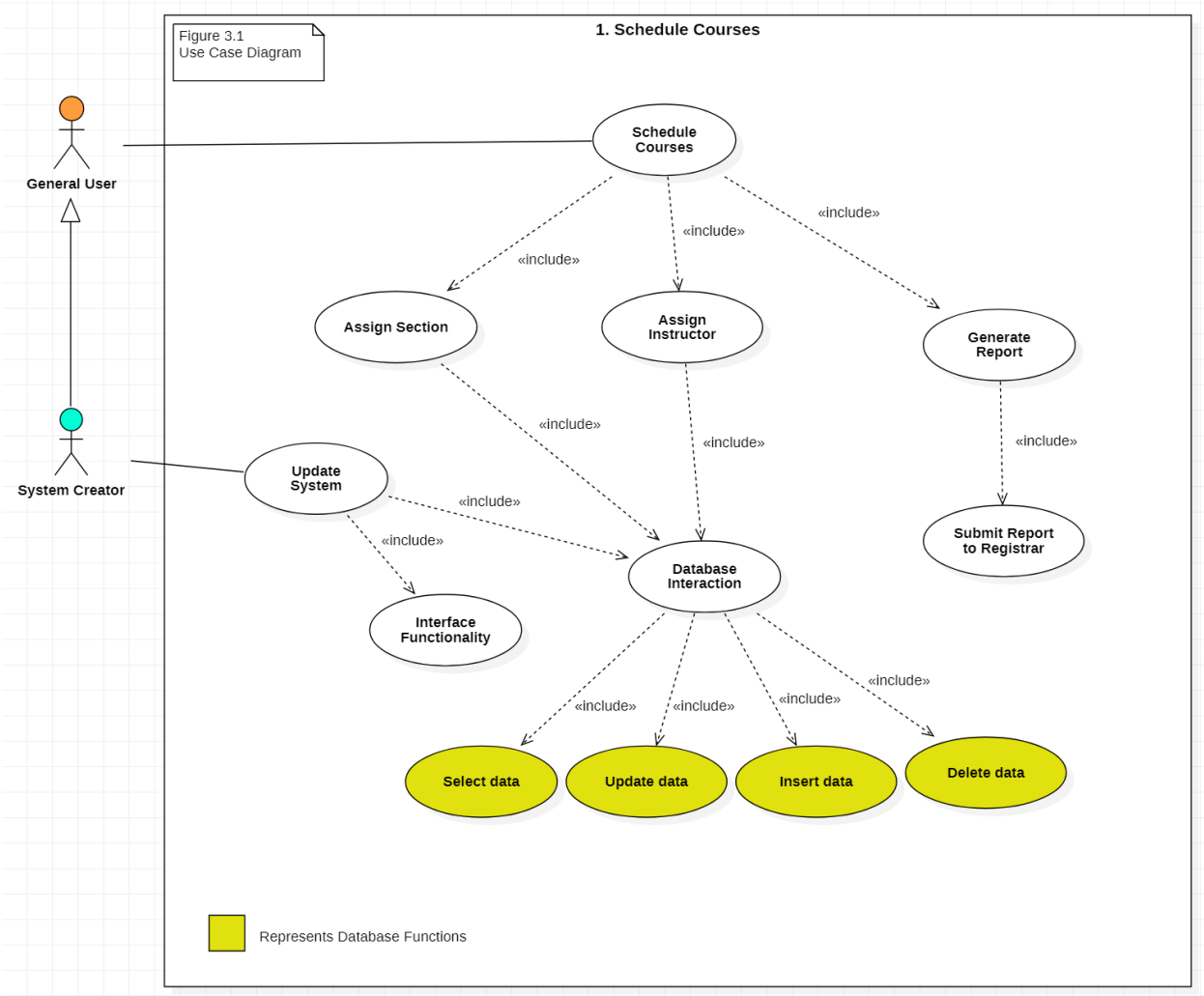

Figure 3.1

4. Analysis Class Diagrams

The following Analysis Class Diagrams visually represents a use case's behavior, showing both the participating classes and software behaviors. Like many other UML diagrams, Analysis Class Diagrams depict how stereotyped objects talk to each other.

4.1. General User

Figure 4.1 below shows all the classes a General User will interact with. A General User will be interacting with the Main Excel Page only, also called the Boundary. These Boundaries will report to other classes, called Control, when the General User has made a choice or decision. Finally, those Control elements will interact with an Entity, which will be the Database.

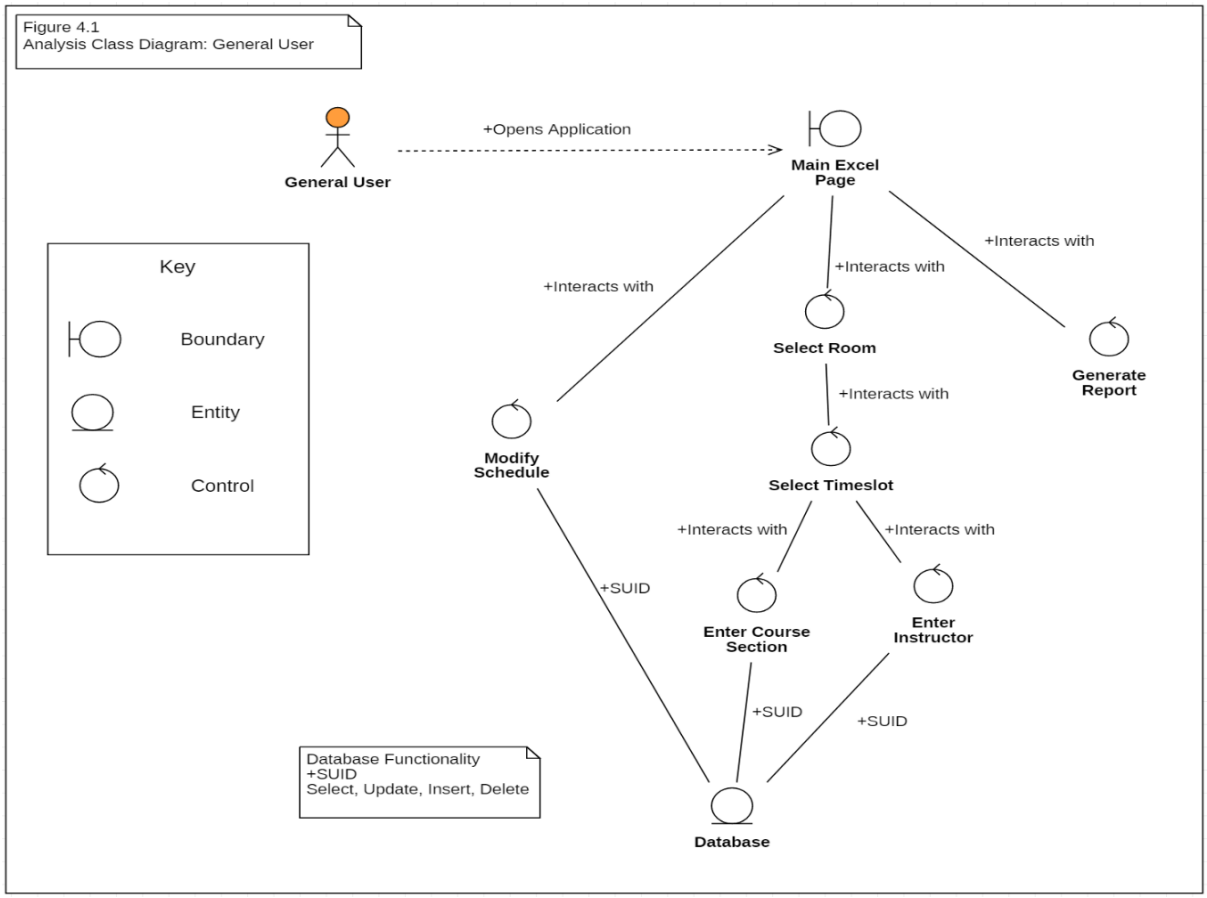

Figure 4.1

4.2. System Creator

Figure 4.2 below shows all the classes a System Creator will interact with. Note that the System Creator can also be a General User; refer to **Figure 4.1** for the Analysis Class Diagram for a General User. For a System Creator, the Boundaries will be either the C# Console or Database Console depending on the role of the System Creator: Excel Team or Database Team. They will each have their respective Entities and Control elements.

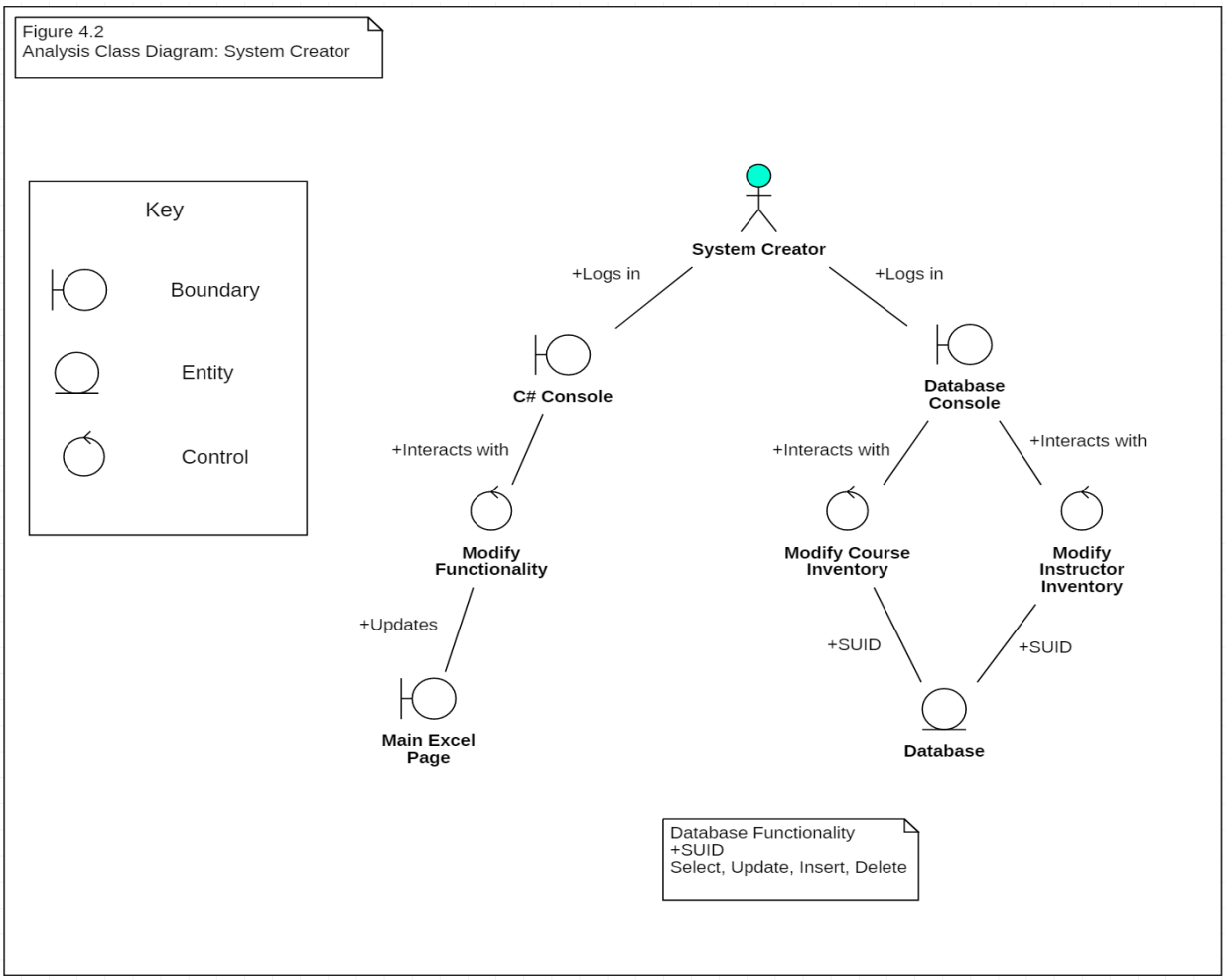

Figure 4.2

5. Statechart Diagram

The following Statechart Diagram is used to show the states of an object during its lifetime and how different events will alter those states. **Figure 5.1** below models the lifetime of the ***Course Scheduler*** application for a General User. The two states are Scheduling and Done Scheduling, both which complement each other until the General User believes that he/she has finally completed the scheduling process.

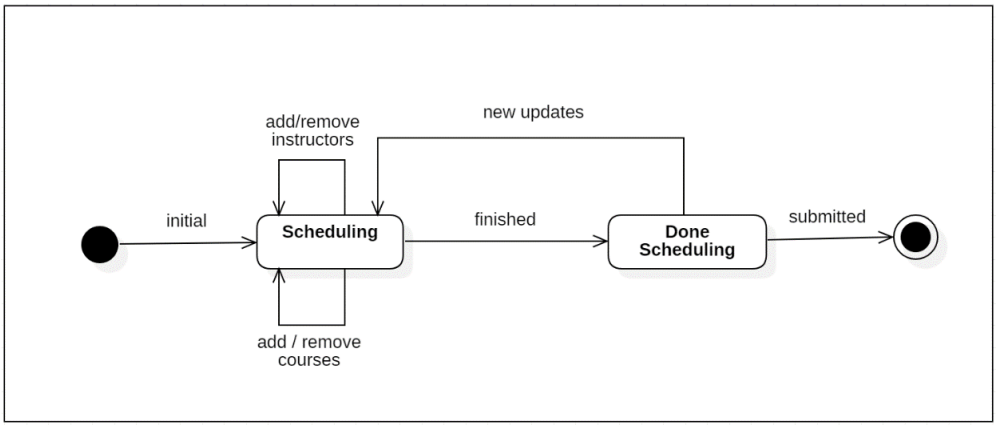

Figure 5.1

6. Class Diagram

The following Class Diagram describes the structure of the C# program by showing the classes and the relationships among other classes. **Figure 6.1** below gives the static view of the ***Course Scheduler*** application. Excel_Functionalities has an unfilled diamond connected to Excel_Workbook because this is an Aggregation, meaning it is part of the Excel_Workbook. DB_Functionalities has a filled diamond connected with DB_Connector because without this class, DB_Functionalities is practically useless without being connected.

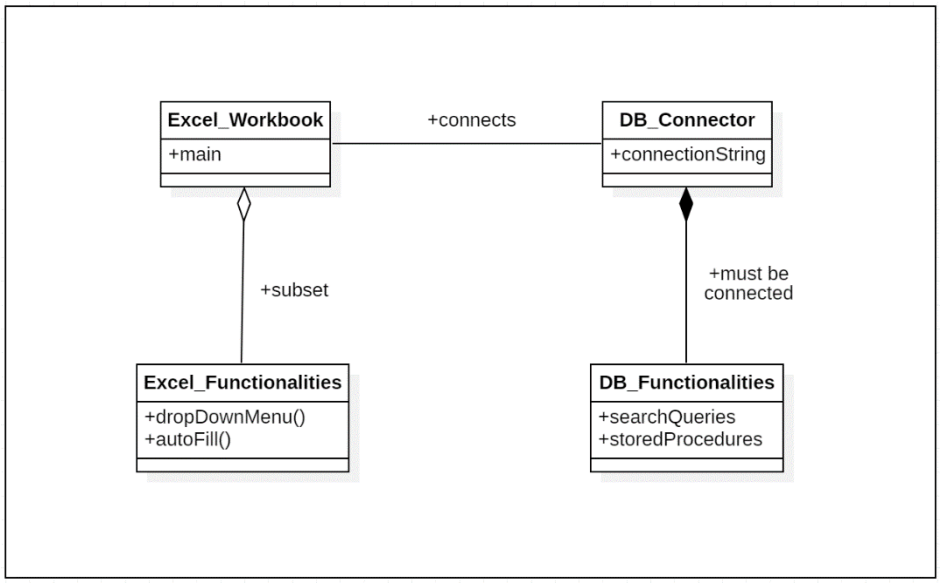

Figure 6.1

7. Activity Diagram

The following Activity Diagram is used to model system behaviors throughout the ***Course Scheduler*** application. It shows step-by-step activities a user can perform while using this application. **Figure 7.1** below depicts a flowchart-like diagram that illustrates the logical paths a process follows based on various conditions. A General User begins the system which has a split, a decision, that may continue as a General User or becomes a System Creator. The two roles here are separated by a vertical line, called a Swimlane, to differentiate which Actors are performing which functions in the system.

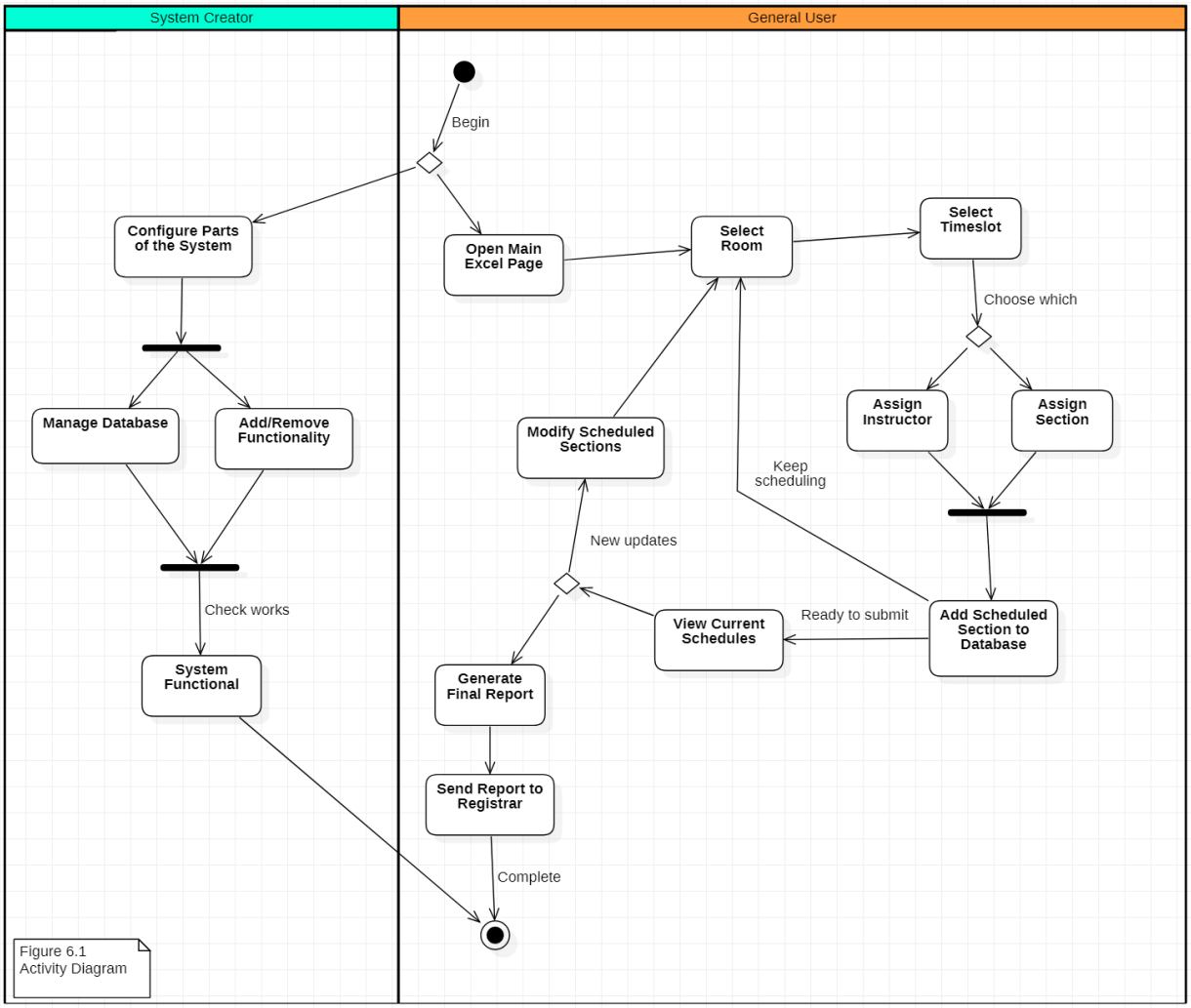

Figure 7.1

8. Sequence Diagrams

The following Sequence Diagrams describe the dynamic interactions between objects. Each diagram below is of a particular functionality and shows the object communication with each other in terms of a temporal sequence of messages.

8.1. Assign Course Section

The following Sequence Diagram shows the **Assign Course Section** process. **Figure 8.1** below shows that the user input gets read in from the Excel Interface and passes it to the Main Console which checks the Database for validity. The Database gets updated and returns reply messages back to the Main Console to notify the user of the status based on their input.

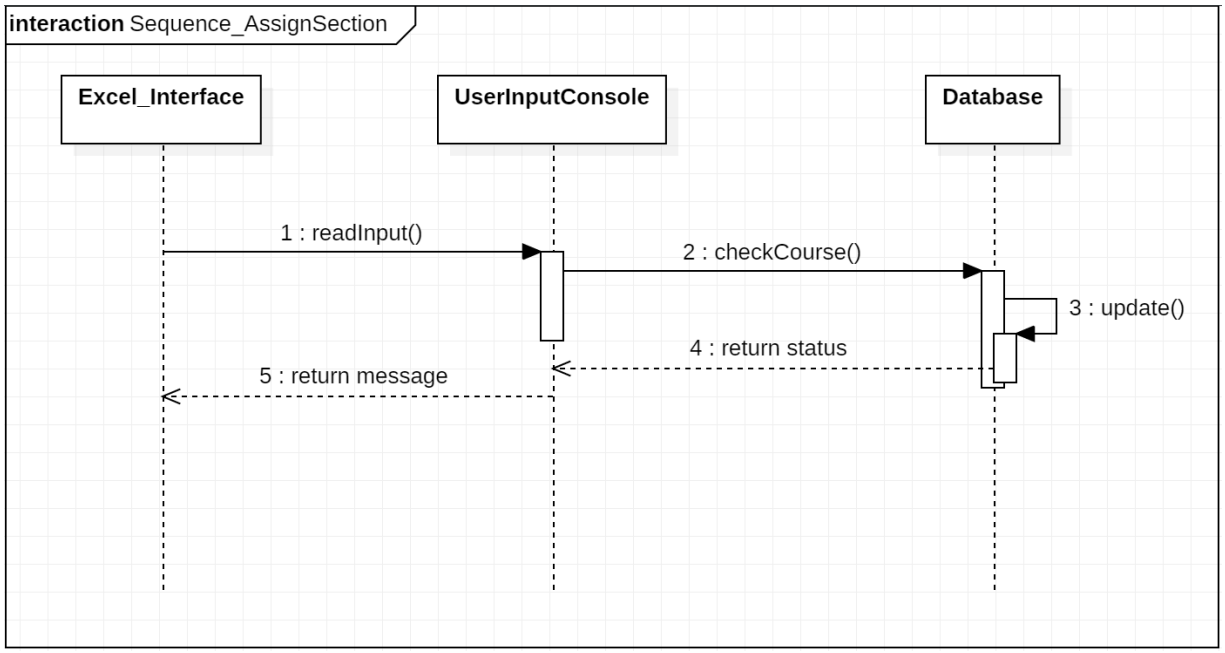

Figure 8.1

8.2. Assign Instructor

The following Sequence Diagram shows the ***Assign Instructor*** process. **Figure 8.2** below shows that the user input gets read in from the Excel Interface and passes it to the Main Console which checks the Database for validity. The Database gets updated, after checking some criteria, and returns reply messages back to the Main Console to notify the user of the status based on their input.

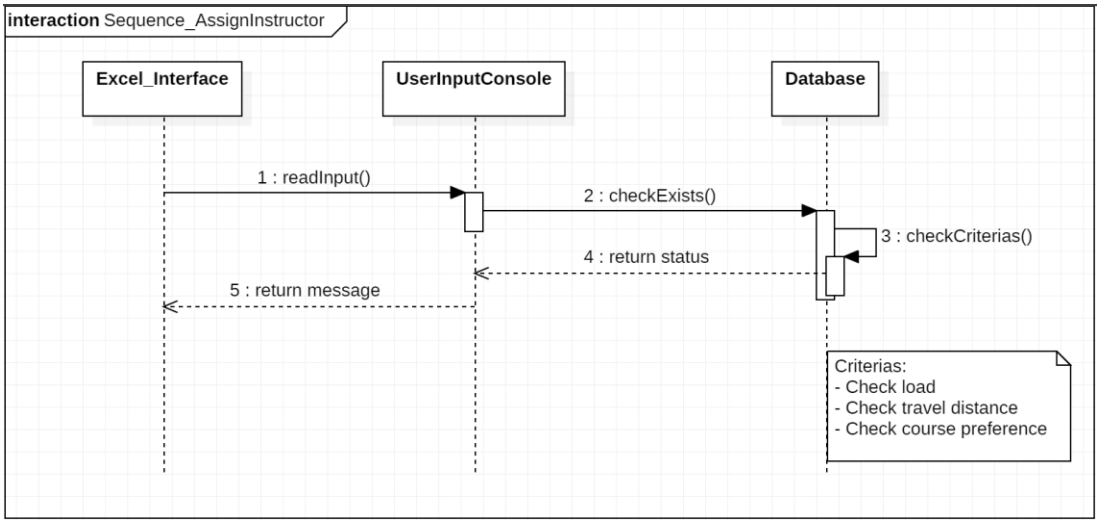

Figure 8.2

8.3. Automatic Suggestion of Course

The following Sequence Diagram shows the **Automatic Suggestion of Course** process. **Figure 8.3** below shows that the user input gets read in from the Excel Interface and passes it to the Main Console which checks the Database for validity. The Database queries for courses that partially or fully match the user input and returns reply messages back to the Main Console. Finally, the Excel Interface displays those returned courses in the dropdown menu.

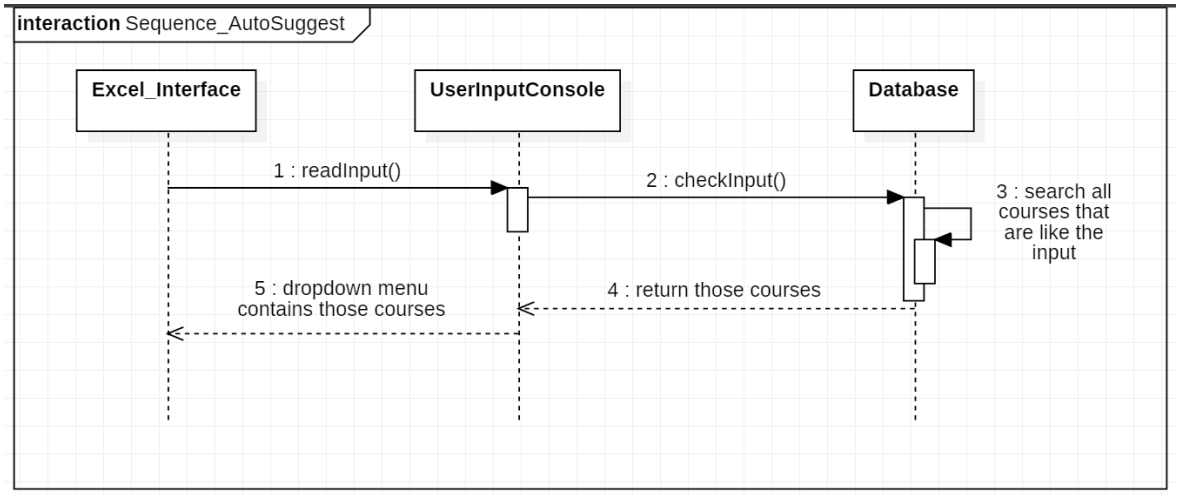

Figure 8.3

8.4. Generate Report

The following Sequence Diagram shows the ***Generate Report*** process. **Figure 8.4** below shows that the user presses a button from the Excel Interface and passes it to the Main Console which will call functions from the database to return the current scheduled courses and format it into a document report.

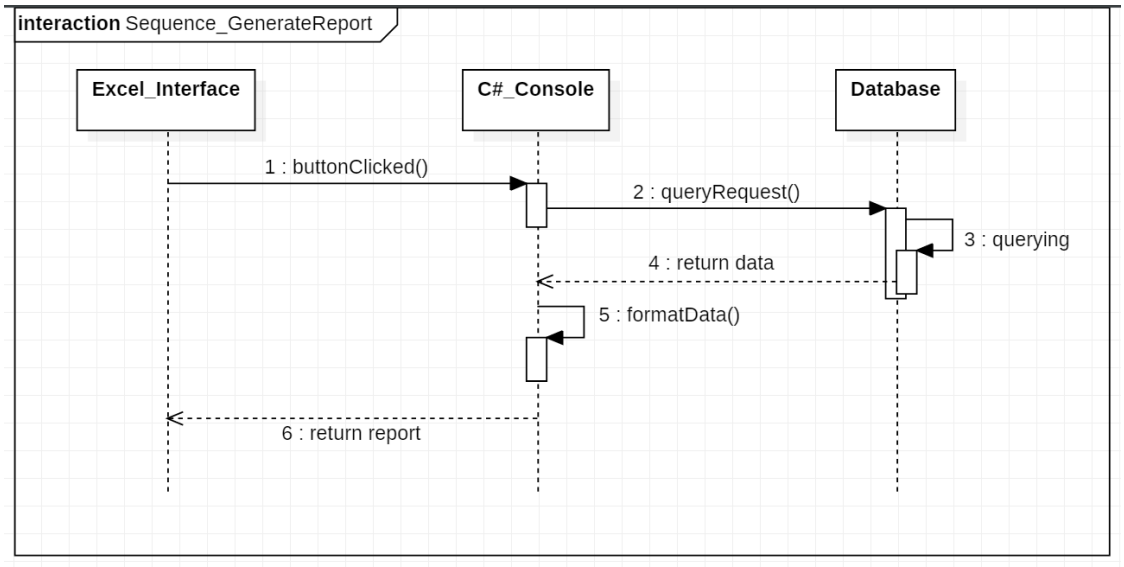

Figure 8.4

8.5. Update Database

The following Sequence Diagram shows the ***Update Database*** process. **Figure 8.5** below shows that a user can import data in the Excel Interface. The Main Console may convert the file to a CSV and update the Database from there. Manual changes can also be done; for example, a new course needs to be added to the database.

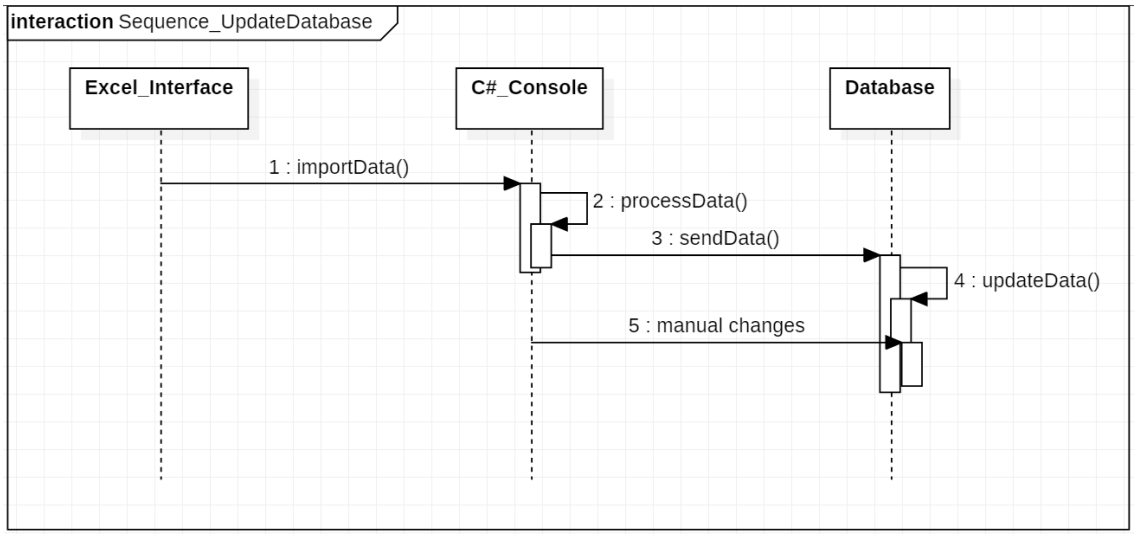

Figure 8.5

9. Entity-Relationship Diagram

The following Entity-Relationship Diagram shows the relationships among the entity sets that are stored in the ***Course Scheduler*** application database. An entity, represented as rectangles in **Figure 9.1** below, is just an object that we want to model information about, similar to a class. The diamonds are relationships between entities. The characters ‘M’ and ‘1’ represents the type of relationship. Having ‘M’ and ‘M’ on both sides means a many-to-many relationship. For example, Instructors can teach many Sections; similarly, Sections can be taught by many Instructors. There are also many-to-one relationships. For example, a Course can have many Sections, but a Section can only belong to one Course.

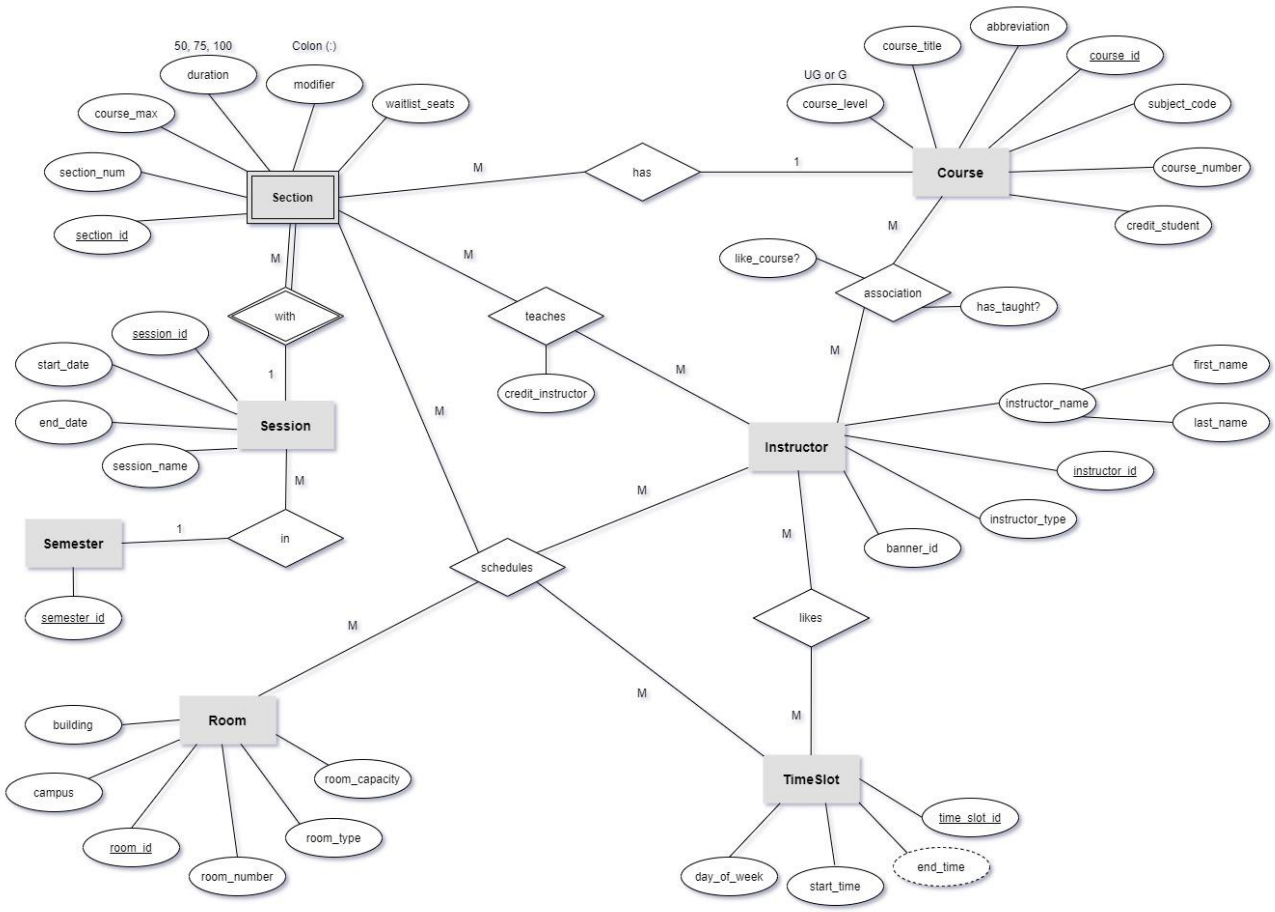

Figure 9.1

10. Revision History

Version	Date	Updated By	Comments
1.0	***03/14/2020***	***Steven Jiang***	Initial document creation
1.1	***03/31/2020***	***Steven Jiang***	Revision for grading